

CS 550 Operating Systems

Spring 2026

Semaphores

Why do we need semaphores?

- Problem with lock
 - ensures mutual exclusion, but no execution order
- Problem with condition variable
 - needs extra effort to track the shared resource
- Producer-consumer problem: need to enforce execution order
 - producer: create resources
 - consumer: use resources
 - bounded buffer between them, which needs to be tracked
 - Execution order: producer waits if buffer full, consumer waits if buffer empty

Semaphore definition

- A synchronization variable that contains an integer value
- Can't access this integer value directly
- Must initialize to some value
 - `sem_init (sem_t *s, int pshared, unsigned int value)`
- Has two operations to manipulate this integer
 - `sem_wait` (or `down()`, `P()`)
 - `sem_post` (or `up()`, `V()`)

```
int sem_wait(sem_t *s) {  
    decrement the value of semaphore s by one  
    wait if value of semaphore s is negative  
}
```

```
int sem_post(sem_t *s) {  
    increment the value of semaphore s by one  
    if there are one or more threads waiting, wake one  
}
```

Semaphore features

- `sem_wait()` will decrement the value of the semaphore variable by 1, and the call will either
 - return right away (if the resulting semaphore value is not negative)
 - in other words, the value of semaphore was 1 or higher when we called `sem_wait()`
 - cause the caller to suspend execution waiting for a subsequent post (if the resulting semaphore value is negative)
- Multiple calling threads may call into `sem_wait()`, and thus all be queued waiting to be woken
- `sem_post()` does not wait. Rather, it simply increments the value of semaphore and then, if there is any thread waiting, wakes one of them up.
- Value of semaphore,
 - when **non-negative**, is equal to the **available amount of resource that the threads are contending for**.
 - when **negative**, is equal to the **number of waiting threads**.

Binary semaphores as locks

1

```
sem_t m;
sem_init(&m, 0, X); // initialize semaphore to X; what should X be?
```

```
sem_wait(&m);
// critical section here
sem_post(&m);
```

Value	Thread 0	State	Thread 1	State
1		Running		Ready
1	call sem_wait()	Running		Ready
0	sem_wait() returns	Running		Ready
0	(crit sect: begin)	Running		Ready
0	Interrupt; Switch→T1	Ready		Running
0		Ready	call sem_wait()	Running
-1		Ready	decrement sem	Running
-1		Ready	(sem<0)→sleep	Sleeping
-1		Running	Switch→T0	Sleeping
-1	(crit sect: end)	Running		Sleeping
-1	call sem_post()	Running		Sleeping
0	increment sem	Running		Sleeping
0	wake(T1)	Running		Ready
0	sem_post() returns	Running		Ready
0	Interrupt; Switch→T1	Ready		Running
0		Ready	sem_wait() returns	Running
0		Ready	(crit sect)	Running
0		Ready	call sem_post()	Running
1		Ready	sem_post() returns	Running

Table 31.2: Thread Trace: Two Threads Using A Semaphore

Shall we use semaphore or mutex for locking?

- Binary semaphore and mutex are semantically similar (when using in the exclusive access scenario), but
 - Mutex can be released only by thread that had acquired it, while one can signal semaphore (wait or post) from any thread (or process)
 - so semaphores are more suitable for some synchronization problems like producer-consumer problem, but not mutual exclusion.
 - Semaphores are heavier.

Semaphores as condition variables

The parent waiting child thread example, again:

```
sem_t s;

void *
child(void *arg) {
    printf("child\n");
    sem_post(&s); // signal here: child is done
    return NULL;
}

int
main(int argc, char *argv[]) {
    sem_init(&s, 0, X); // what should X be? 0
    printf("parent: begin\n");
    pthread_t c;
    Pthread_create(c, NULL, child, NULL);
    sem_wait(&s); // wait here for child
    printf("parent: end\n");
    return 0;
}
```

The producer/consumer problem

```
int buffer[MAX];
int fill = 0;
int use = 0;
```

```
void put(int value) {
    buffer[fill] = value;
    fill = (fill + 1) % MAX;
}
```

```
int get() {
    int tmp = buffer[use];
    use = (use + 1) % MAX;
    return tmp;
}
```

`sem_t empty;` *The buffer is empty at the beginning. What should be the initial value of “empty”?*

`sem_t full;` *The buffer is empty at the beginning. What should be the initial value of “full”?*

```
void *producer(void *arg) {
P1     int i;
P2     for (i = 0; i < loops; i++) {
P3         sem_wait(&empty);
P4         put(i);
P5         sem_post(&full);
        }
}
```

```
void *consumer(void *arg) {
C1     int i, tmp = 0;
C2     while (tmp != -1) {
C3         sem_wait(&full);
C4         tmp = get();
C5         sem_post(&empty);
C6         printf("%d\n", tmp);
    }
}
```


The producer/consumer problem

```
int buffer[MAX];
int fill = 0;
int use = 0;
```

MAX = 1 ...



MAX > 1 ...



Why?

```
void put(int value) {
    buffer[fill] = value;
    fill = (fill + 1) % MAX;
}
```

```
int get() {
    int tmp = buffer[use];
    use = (use + 1) % MAX;
    return tmp;
}
```

`sem_t empty;` *The buffer is empty at the beginning. What should be the initial value of "empty"? MAX*
`sem_t full;` *The buffer is empty at the beginning. What should be the initial value of "full"? 0*

```
void *producer(void *arg) {
P1     int i;
P2     for (i = 0; i < loops; i++) {
P3         sem_wait(&empty);
P4         put(i);
P5         sem_post(&full);
        }
}
```

```
void *consumer(void *arg) {
C1     int i, tmp = 0;
C2     while (tmp != -1) {
C3         sem_wait(&full);
C4         tmp = get();
C5         sem_post(&empty);
C6         printf("%d\n", tmp);
    }
}
```

Let's look at this fix ...

```
sem_t empty;  
sem_t full;  
sem_t mutex;
```

```
void *producer(void *arg) {  
    int i;  
    for (i = 0; i < loops; i++) {  
        sem_wait(&mutex);  
        sem_wait(&empty);  
        put(i);  
        sem_post(&full);  
        sem_post(&mutex);  
    }  
}
```

```
void *consumer(void *arg) {  
    int i;  
    for (i = 0; i < loops; i++) {  
        sem_wait(&mutex);  
        sem_wait(&full);  
        int tmp = get();  
        sem_post(&empty);  
        sem_post(&mutex);  
        printf("%d\n", tmp);  
    }  
}
```

Any problems?

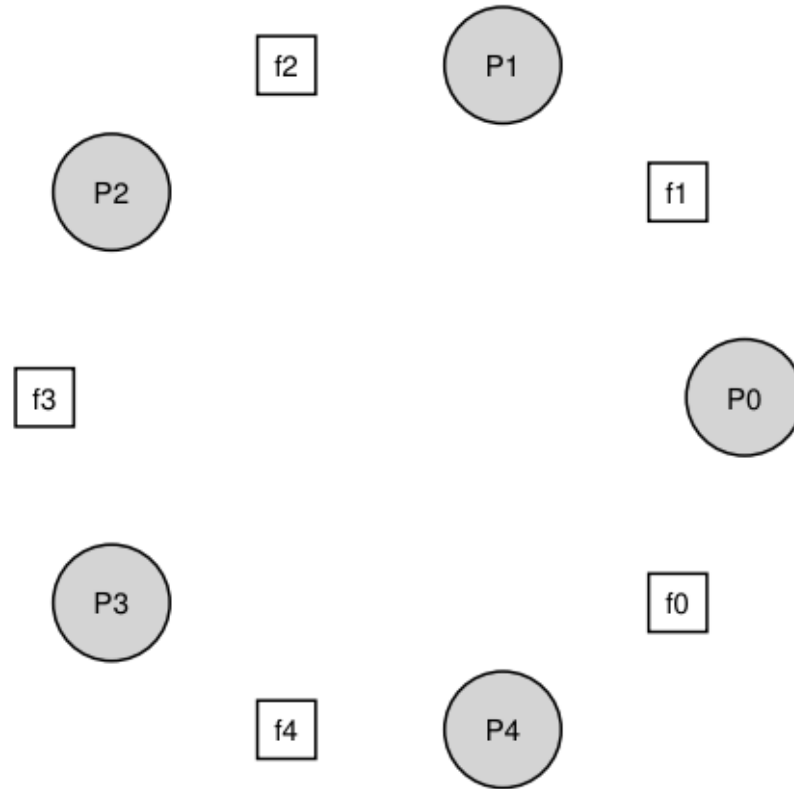
The working solution

```
sem_t empty;  
sem_t full;  
sem_t mutex;
```

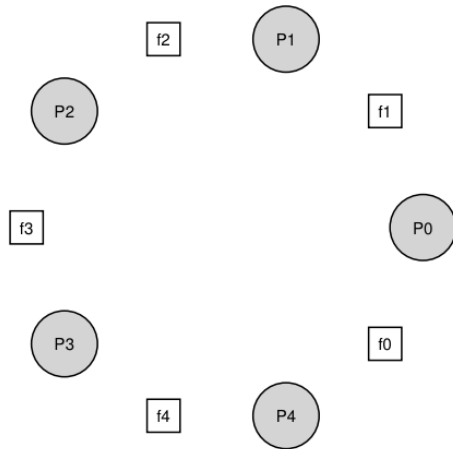
```
void *producer(void *arg) {  
    int i;  
    for (i = 0; i < loops; i++) {  
        sem_wait(&empty);  
        sem_wait(&mutex);  
        put(i);  
        sem_post(&mutex);  
        sem_post(&full);  
    }  
}
```

```
void *consumer(void *arg) {  
    int i;  
    for (i = 0; i < loops; i++) {  
        sem_wait(&full);  
        sem_wait(&mutex);  
        int tmp = get();  
        sem_post(&mutex);  
        sem_post(&empty);  
        printf("%d\n", tmp);  
    }  
}
```

The dining philosophers



The dining philosophers



Basic loop of each philosopher:

```
while (1) {  
    think();  
    getforks();  
    eat();  
    putforks();  
}
```

Helper functions:

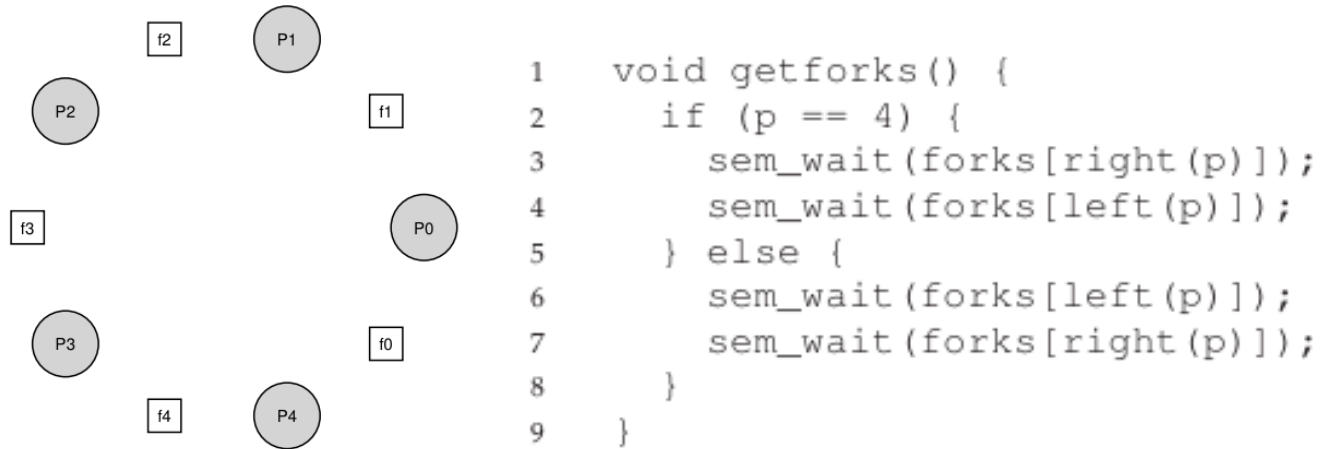
```
int left(int p) { return p; }  
int right(int p) { return (p + 1) % 5; }
```

Let's look at this solution:

```
1 void getforks() {  
2     sem_wait(forks[left(p)]);  
3     sem_wait(forks[right(p)]);  
4 }  
5  
6 void putforks() {  
7     sem_post(forks[left(p)]);  
8     sem_post(forks[right(p)]);  
9 }
```

Any problem?

The dining philosophers



How is the deadlock problem prevented by this solution?

- Circular wait [Breaking the dependency](#)
- Hold-and-wait
- No preemption
- Mutual exclusion